

CO - 4 ASSESSMENT TOOL -3

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA0619
Subject Name : DAA

Q17 : Given a string and dictionary, solve Word Break and compare with Word Wrap problem using DP. Analyze differences in objective functions. Evaluate complexity of both problems. Discuss optimal substructure and overlapping sub problems. Compare outputs and constraints. Generate insights on text processing optimization.

Sol :

Source code :

```
def word_break(s, dictionary):  
    n = len(s)  
    dp = [False] * (n + 1)  
    dp[0] = True  
    for i in range(1, n + 1):  
        for j in range(i):  
            if dp[j] and s[j:i] in dictionary:  
                dp[i] = True  
                break  
    return dp[n]  
  
def word_wrap(words, width):  
    n = len(words)  
    dp = [float('inf')] * (n + 1)  
    dp[0] = 0  
    for i in range(1, n + 1):  
        length = 0
```

```

for j in range(i, 0, -1):

    length += len(words[j - 1])

    spaces = i - j

    if length + spaces <= width:

        cost = (width - length - spaces) ** 2

        dp[i] = min(dp[i], dp[j - 1] + cost)

    else:

        break

return dp[n]

s = "applepie"

dictionary = {"apple", "pie"}

words = ["This", "is", "a", "simple", "text"]

print("Word Break:", word_break(s, dictionary))

print("Minimum Word Wrap Cost:", word_wrap(words, 10))

```

Aim

To solve the **Word Break** and **Word Wrap** problems using Dynamic Programming and compare their objectives, complexity, constraints, and optimal substructure.

Objective

- Determine whether a string can be divided into valid dictionary words.
- Minimize the cost of arranging words within a fixed line width.
- Study optimal substructure and overlapping subproblems.
- Compare both DP-based text processing problems.

3. Word Break Problem

Input

- A string.
- A dictionary containing valid words.

Example:

```
String = "applepie"  
Dictionary = {"apple", "pie"}
```

Output

True

because:

apple + pie

forms the given string.

Objective Function

The objective is simply to determine:

$DP[n] = \text{True}$ if the complete string can be formed using dictionary words.

4. Word Wrap Problem

Input

- A list of words.
- Maximum line width.

Example:

```
Words = ["This", "is", "a", "simple", "text"]  
Width = 10
```

Output

The minimum total wrapping cost.

The code uses:

$\text{Cost} = (\text{Width} - \text{used space})^2$

The objective is to **minimize the total cost of extra spaces**.

5. Optimal Substructure

Word Break

If a string can be divided successfully at position j , then:

First part $\rightarrow$ already solvable

Second part $\rightarrow$ valid dictionary word

Therefore, the solution to a larger problem depends on solutions to smaller prefixes.

Word Wrap

The optimal arrangement of the first i words can be obtained by considering different possible starting positions for the last line.

Thus, the optimal solution is built from optimal solutions of smaller prefixes.

6. Overlapping Subproblems

Both problems have overlapping subproblems.

Word Break

The same prefixes can be checked multiple times.

DP stores the results in:

$dp[i]$

to avoid repeated computation.

Word Wrap

Different line arrangements may repeatedly calculate the best arrangement for the same prefix of words. DP stores these values and reuses them.

7. Complexity

Word Break

For a string of length n :

Time= $O(n^2)$ Time = $O(n^2)$ Space= $O(n)$ Space = $O(n)$

Word Wrap

For n words:

Time= $O(n^2)$ Time = $O(n^2)$ Space= $O(n)$ Space = $O(n)$

Both algorithms have quadratic time complexity because they consider different partitions of the input.

8. Comparison

Feature	Word Break	Word Wrap
Input	String + dictionary	Words + line width
Objective	Check valid segmentation	Minimize line cost
Output	True/False	Minimum cost
DP State	Valid prefix	Minimum wrapping cost
Time	$O(n^2)$	$O(n^2)$
Space	$O(n)$	$O(n)$
Main Constraint	Words must exist in dictionary	Words must fit line width

9. Insights on Text Processing Optimization

Both problems demonstrate how **Dynamic Programming avoids repeated calculations** by storing solutions to smaller subproblems. Word Break focuses on **valid segmentation**, while Word Wrap focuses on **cost optimization**. DP is therefore useful in text processing where many possible word boundaries or arrangements must be evaluated efficiently.

Result

Word Break and Word Wrap were successfully implemented using Dynamic Programming. Both problems exhibit **optimal substructure and overlapping subproblems**, but their objective functions differ: Word Break checks **feasibility**, whereas Word Wrap minimizes a **layout cost**.

Output Screenshot :

```
# Word Break using DP

def word_break(s, dictionary):
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in dictionary:
                dp[i] = True
                break

    return dp[n]

# Word Wrap using DP

def word_wrap(words, width):
    n = len(words)
    dp = [float('inf')] * (n + 1)
    dp[0] = 0

    for i in range(1, n + 1):
        length = 0

        for j in range(i, 0, -1):
            length += len(words[j - 1])

            spaces = i - j
            if length + spaces <= width:
                cost = (width - length - spaces)
                dp[i] = min(dp[i], dp[j - 1] + cost)

            else:
                break

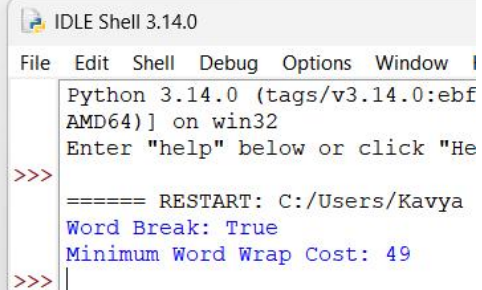
    return dp[n]

# Example

s = "applepie"
dictionary = {"apple", "pie"}

words = ["This", "is", "a", "simple", "text"]

print("Word Break:", word_break(s, dictionary))
print("Minimum Word Wrap Cost:", word_wrap(words, width))
```



The screenshot shows the IDLE Shell 3.14.0 interface. The menu bar includes File, Edit, Shell, Debug, Options, and Window. The shell displays the following output:

```
Python 3.14.0 (tags/v3.14.0:ebf
AMD64)] on win32
Enter "help" below or click "He
>>>
===== RESTART: C:/Users/Kavya
Word Break: True
Minimum Word Wrap Cost: 49
>>> |
```